

Legacy ETL Compiler

Mark London

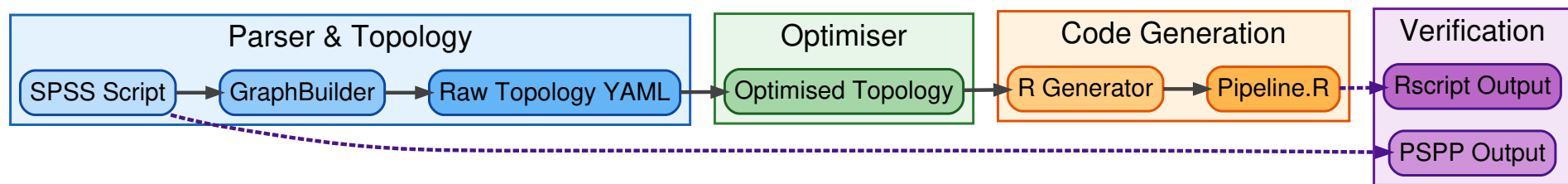
2026-10-03



Overview

- Purpose: migrate SPSS logic into R pipelines
- Verification & validation cycle of output
This is a proof of concept to explore the value potential of developing it into full product.

Architecture



- PSPP is a free Linux version of SPSS. Whilst not identical, it accepts most SPSS syntax.

What is the legacy-etl-compiler system?

- Takes an existing SPSS pipeline and converts it into another language (e.g., R).
- Uses an intermediate representation to make the compiler deterministic, testable, and optimisable.
- The key intermediate representation is a State Machine built from the original script.

What is a State Machine?

- A mathematical model of a program where each operation explicitly transitions the system to a new, immutable state.
- Instead of mutating variables, we create new versions at each step (e.g., `x1` , `x2` , `x3`).
- Each step is deterministic, making complex, messy legacy pipelines significantly easier to reason about.

Order Matters

Side-by-Side Comparison

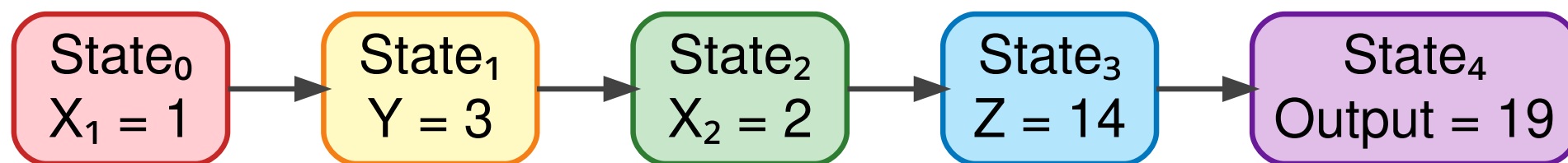
Original Order

```
1 X <- 1      → X = 1
2 Y <- X * 3   → Y = 3
3 X <- X + 1   → X = 2
4 Z <- X * 7   → Z = 14
5 sum(X, Y, Z)
6 Result = 19
```

Modified Order

```
1 X <- 1      → X = 1
2 Z <- X * 7   → Z = 7
3 X <- X + 1   → X = 2
4 Y <- X * 3   → Y = 6
5 sum(X, Y, Z)
6 Result = 15
```

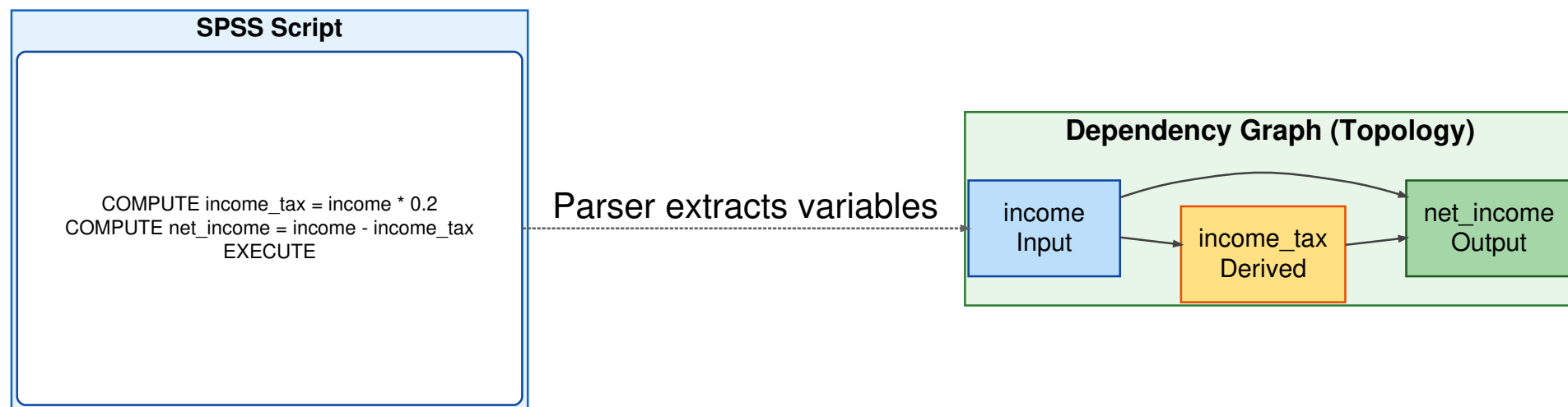
State Machine View (Same Logic)



State Machine (Compiler-Level)

- Operations become nodes in a directed acyclic graph.
- States are the intermediate datasets produced by each node.
- The compiler can then optimise, reorder, and validate these transformations.

Example: Dependency Graph



Why not translate directly to R?

- The Syntax Trap: Direct translation maps SPSS syntax directly to R syntax. This fails because the two languages have fundamentally different execution models.
- Hidden Side Effects: SPSS is highly procedural and relies on a mutable “active dataset.” R leans towards functional dataframe manipulation.
- Brittle Output: Without an intermediate logical layer, the resulting R code is often unreadable, unoptimised, and extremely difficult to test for true equivalence.

Why via a State Machine?

- A State Machine models the true underlying *territory* of the data lifecycle, independent of the *map* (the syntax) of either language.
- It makes dependencies explicitly clear and eliminates hidden side effects.
- It safely allows the compiler to reorder, optimise, and mathematically verify transformations while ensuring behavioural equivalence.

Verification Artifacts

The folder `new_dist/verification` captures the full verification chain for the compiler run.

- 1 - `00_original_spss.txt`: original SPSS/PSPP source script
- 2 - `01_source_verification.txt`: PSPP execution status and output
- 3 - `02_raw_topology.yaml`: compiler's initial topology (graph) for the source
- 4 - `03_optimized_topology.yaml`: topology after compiler optimizations
- 5 - `04_generated_code.R`: generated R pipeline
- 6 - `05_target_verification.txt`: R execution status and output

00_original_spss.txt (Source)

```
1  * Hello World Pipeline.
2
3  * 1. LOAD .
4  GET DATA
5      /TYPE=TXT
6      /FILE='hello.csv'
7      /ARRANGEMENT=DELIMITED
8      /DELIMITERS=', '
9      /FIRSTCASE=2
10     /VARIABLES=
11         id F8.0
12         name A10.
13
14 * 2. SORT (The Test Case).
15 SORT CASES BY id.
16
17 * 3. SAVE (Using TRANSLATE to get a readable CSV).
18 SAVE TRANSLATE
19     /OUTFILE='hello_sorted.csv'
20     /TYPE=CSV
21     /REPLACE.
```

01_source_verification.txt (PSPP run)

```
1 Command: pspp hello.sps
2 Status:  Success
3
4 === OUTPUT ===
```

02_raw_topology.yaml (Compiled Topology)

```
1 # Pipeline Topology: 3 Operations
2 -----
3 Operation: op_001_load
4   Type:    LOAD_CSV
5   Inputs:  []
6   Outputs: ['source_hello.csv']
7   Params: {'filename': 'hello.csv', 'format': 'TXT', 'schema': ['id', 'name'], 'skip_rows': 1}
8
9 Operation: op_002_sort
10  Type:    SORT_ROWS
11  Inputs:  ['source_hello.csv']
12  Outputs: ['ds_001_sorted']
13  Params: {'keys': 'id', 'order': 'ascending'}
14
15 Operation: op_003_save
16  Type:    SAVE_BINARY
17  Inputs:  ['ds_001_sorted']
18  Outputs: ['file_hello_sorted.csv']
19  Params: {'filename': 'hello_sorted.csv'}
```


03_optimized_topology.yaml (Optimized Topology)

```
1 # Pipeline Topology: 3 Operations
2 -----
3 Operation: op_001_load
4   Type:    LOAD_CSV
5   Inputs:  []
6   Outputs: ['source_hello.csv']
7   Params: {'filename': 'hello.csv', 'format': 'TXT', 'schema': ['id', 'name'], 'skip_rows': 1}
8
9 Operation: op_002_sort
10  Type:    SORT_ROWS
11  Inputs:  ['source_hello.csv']
12  Outputs: ['ds_001_sorted']
13  Params: {'keys': 'id', 'order': 'ascending'}
14
15 Operation: op_003_save
16  Type:    SAVE_BINARY
17  Inputs:  ['ds_001_sorted']
18  Outputs: ['file_hello_sorted.csv']
19  Params: {'filename': 'hello_sorted.csv'}
```

04_generated_code.R (Generated R)

```
1 # Auto-Generated ETL Script
2 # Generated by: V&V Compiler
3 # Dialect: Tidyverse
4
5 library(tidyverse)
6 library(lubridate)
7 library(haven)
8
9 # Op: op_001_load
10 source_hello.csv <- read_csv("hello.csv")
11
12 # Op: op_002_sort
13 ds_001_sorted <- source_hello.csv %>% arrange(id)
14
15 # Op: op_003_save
16 write_csv(ds_001_sorted, "hello_sorted.csv")
```

05_target_verification.txt (R run)

```
1 Command: Rscript dist/pipeline.R
2 Status:  Success
3
4 === OUTPUT ===
5
6 — Column specification —————
7 Delimiter: ","
8 chr (1): name
9 dbl (1): id
```

Supported Features

```
1 Data Loading
2 - GET DATA /TYPE=TXT /FILE='filename.csv' /DELIMITERS=', '
3   - Loads CSV files with comma delimiters - Generates `read_csv()` in R
4
5 - GET DATA /TYPE=SAV /FILE='filename.sav'
6   - Loads SPSS .sav files - Generates `read_sav()` in R
7
8 Data Manipulation
9 - SORT CASES BY variable(s) (A/D)
10 - COMPUTE target = expression
11   - Basic arithmetic and logical expressions
12   - Variable assignments
13
14 - IF (condition) target = expression
15   - Generates `mutate()` with `if_else()` in R
16
17 - RECODE source_vars (...) INTO target_vars
18   - Variable recoding with mapping rules
```

Cont

```
1 Data Filtering
2 - SELECT IF (condition)
3   - Row filtering based on conditions
4   - Generates `filter()` in R
5
6 Data Joining
7 - MATCH FILES /FILE=* /TABLE='file' /BY key
8   - Inner and left joins
9   - Multiple key variables
10  - Generates `inner_join()` and `left_join()` in R
11
12 Aggregation
13 - AGGREGATE /BREAK=group_vars /target=func(source)
14   - Grouped aggregations
15   - Functions: MEAN, SUM, MAX, MIN
16   - Generates `group_by()` and `summarise()` in R
17
18 Schema Definition
19 - DATA LIST FREE / var1 (F8.0) var2 (A10)
20   - Variable type definitions
21   - Basic format specifications
22   - Injected into IR schema
23
24 Data Saving
25 - SAVE OUTFILE='filename.sav'
26   - Saves to SPSS .sav format
27   - Generates `write_sav()` in R
28
29 - SAVE OUTFILE='filename.csv'
30   - Saves to CSV format
31   - Generates `write_csv()` in R
```

Supported Functions and Operators

```
1 Mathematical Functions
2 - TRUNC() → floor()
3 - RND() → round()
4 - ABS() → abs()
5 - MOD(a,b) → a %% b
6
7 String Functions
8 - CONCAT() → paste0()
9 - PASTE() → paste0()
10 - STR_C() → str_c()
11
12 Statistical Functions
13 - MEAN() → mean()
14 - LAG() → lag()
15
16 Logical Operators
17 - AND → &
18 - OR → |
19 - <> → !=
20 - = → == (in expressions)
21
22 Conditional Functions
23 - IF_ELSE() → if_else()
24 - CASE_WHEN() → case_when()
25 - NA_IF() → na_if()
```

Cont

```
1 Date Functions
2 - DATE.MDY(m,d,y) → make_date(year=y, month=m, day=d)
3
4 Special Values
5 - $SYSMIS → NA
6
7 Partially Supported Features
8
9 Conditional Logic
10 - DO IF / END IF blocks
11   - Basic structure recognized
12   - May have formatting inconsistencies in generated conditions
13
14 Generic Commands
15 - Unknown SPSS commands parsed as generic nodes
16 - Logged as warnings in generated R code
17 - No functional implementation
```

Unsupported Features

- 1 Data Input
- 2 - Complex DATA LIST formats beyond FREE
- 3 - BEGIN DATA / END DATA inline data blocks (skipped)
- 4 - Other file types beyond TXT and SAV
- 5 - Complex delimiter specifications
- 6
- 7 Advanced Functions
- 8 - Most SPSS-specific functions not in the registry
- 9 - Advanced string manipulation beyond CONCAT
- 10 - Complex date/time operations beyond MDY
- 11 - Missing value patterns beyond \$SYSMIS
- 12
- 13 Control Flow
- 14 - Complex nested IF/DO IF structures
- 15 - LOOP/END LOOP constructs
- 16 - Macro definitions and calls
- 17
- 18 Output
- 19 - Complex SAVE options and formats
- 20 - EXPORT commands
- 21 - PRINT/WRITE statements
- 22
- 23 Statistical Procedures
- 24 - All statistical analysis commands (DESCRIPTIVES, FREQUENCIES, etc.)
- 25 - Parsed as generic nodes with warnings
- 26
- 27 File Management
- 28 - FILE HANDLE definitions
- 29 - Complex file path handling
- 30 - Temporary file management

Questions?

WAS RADAR

Speaker notes